Flight Delay Prediction

36-673 Final Project

Ellie Huang (elliehua) and Nikita Jayaprakash (njayapra)

2026-04-22

Data Prep

Load Libraries and Data

```
# Load data
flights_2023 <- read_csv(here("data", "flights2023.csv")) |>
  clean_names()

flights_2024 <- read_csv(here("data", "flights2024.csv")) |>
  clean_names()

flights_test <- read_csv(here("data", "flights2025.csv")) |>
  clean_names()

flights_test_visible <- read_csv(here("data", "flights2025_visible.csv")) |>
  clean_names()

flights_test_guess <- read_csv(here("data", "flights2025_guess.csv")) |>
  clean_names()

# Build training set
flights_train <- bind_rows(
  flights_2023 |> mutate(data_year = 2023),
  flights_2024 |> mutate(data_year = 2024)
)
```

Data Prep

```
# Cleaning function - apply to all data frames so train and test are consistent
clean_flights <- function(df) {
  df |>
    clean_names() |>
    mutate(
      fl_date = as.Date(fl_date),  # date
      dep_del15 = as.numeric(dep_del15),  # target
      op_unique_carrier = as.factor(op_unique_carrier),  # factors
      origin = as.factor(origin),
      dest = as.factor(dest),
      dep_hour = floor(dep_time / 100),  # time features
      sched_dep_hour = floor(crs_dep_time / 100),  # use this for test rows where dep_time is NA
```

```r
      month = as.factor(month),
      day_of_week = as.factor(day_of_week)
    )
}

flights_train <- clean_flights(flights_train)
flights_test <- clean_flights(flights_test)
flights_test_visible <- clean_flights(flights_test_visible)
flights_test_guess <- clean_flights(flights_test_guess)
```

```r
# Filter for only departures
flights_train <- flights_train |>
  filter(origin == "PIT")

flights_test_visible <- flights_test_visible |>
  filter(origin == "PIT")

flights_test_guess <- flights_test_guess |>
  filter(origin == "PIT")

# Check target distribution
table(flights_train$dep_del15, useNA = "ifany")
```

```
##
##     0     1  <NA>
## 59534 10972   950
```

Exploratory Data Analysis

Missingness

```r
# Missingness
colSums(is.na(flights_train[, c("dep_del15", "dep_time", "crs_dep_time")]))
```

```
##    dep_del15     dep_time crs_dep_time
##          950          950            0
```

```r
flights_train |>
  summarise(across(everything(), ~mean(is.na(.)))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
  filter(pct_missing > 0) |>
  arrange(desc(pct_missing)) |>
  print(n = 30)
```

```
## # A tibble: 28 x 2
##    variable            pct_missing
##    <chr>                     <dbl>
##  1 first_dep_time          0.995
##  2 total_add_gtime         0.995
##  3 longest_add_gtime       0.995
##  4 cancellation_code       0.986
##  5 carrier_delay           0.837
##  6 weather_delay           0.837
##  7 nas_delay               0.837
```

```
##  8 security_delay          0.837
##  9 late_aircraft_delay     0.837
## 10 fl_date                 0.609
## 11 arr_delay               0.0154
## 12 arr_delay_new           0.0154
## 13 arr_del15               0.0154
## 14 arr_delay_group         0.0154
## 15 actual_elapsed_time     0.0154
## 16 air_time                0.0154
## 17 wheels_on               0.0138
## 18 taxi_in                 0.0138
## 19 arr_time                0.0138
## 20 taxi_out                0.0137
## 21 wheels_off              0.0137
## 22 dep_time                0.0133
## 23 dep_delay               0.0133
## 24 dep_delay_new           0.0133
## 25 dep_del15               0.0133
## 26 dep_delay_group         0.0133
## 27 dep_hour                0.0133
## 28 tail_num                0.00158
```

Most of the missingness is in delay + reason columns (carrier delay, weather delay, etc.) that we can't use for prediction anyway. The main thing we should check is whether any pre-departure features we actually plan to use have meaningful gaps.

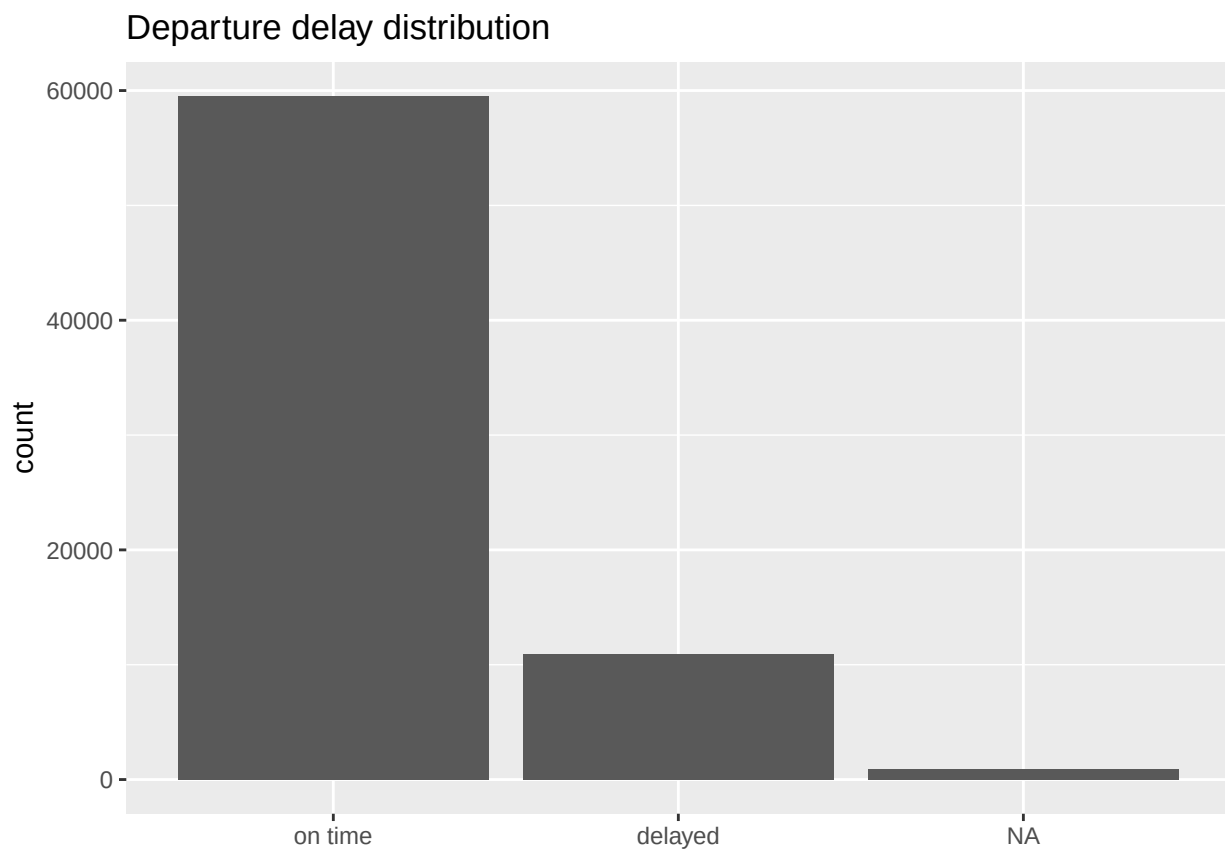
NOTE: add analysis for missingness, see if there's any patterns to missingness (and if we can include missingness in modelling)

Departure Delay Dist

```r
# Target distribution
flights_train |>
  count(dep_del15)
```

```
## # A tibble: 3 x 2
##   dep_del15     n
##       <dbl> <int>
## 1         0 59534
## 2         1 10972
## 3        NA   950
```

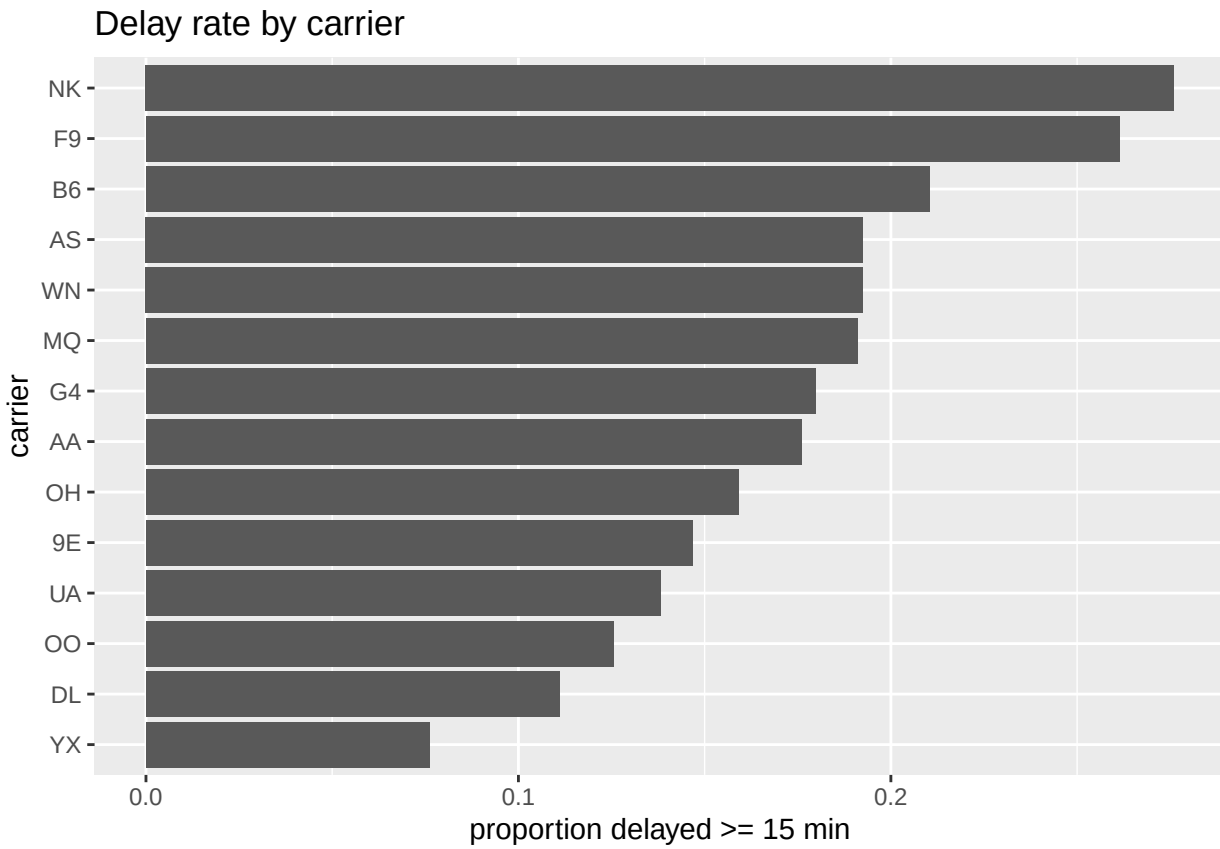
```r
flights_train |>
  ggplot(aes(x = factor(dep_del15, labels = c("on time", "delayed")))) +
  geom_bar() +
  labs(title = "Departure delay distribution", x = NULL, y = "count")
```



The classes are roughly 80/20 on-time vs. delayed. Since AUC is our evaluation metric this imbalance isn't too concerning, but it's worth keeping in mind if we end up picking a classification threshold later.

Carrier Delay

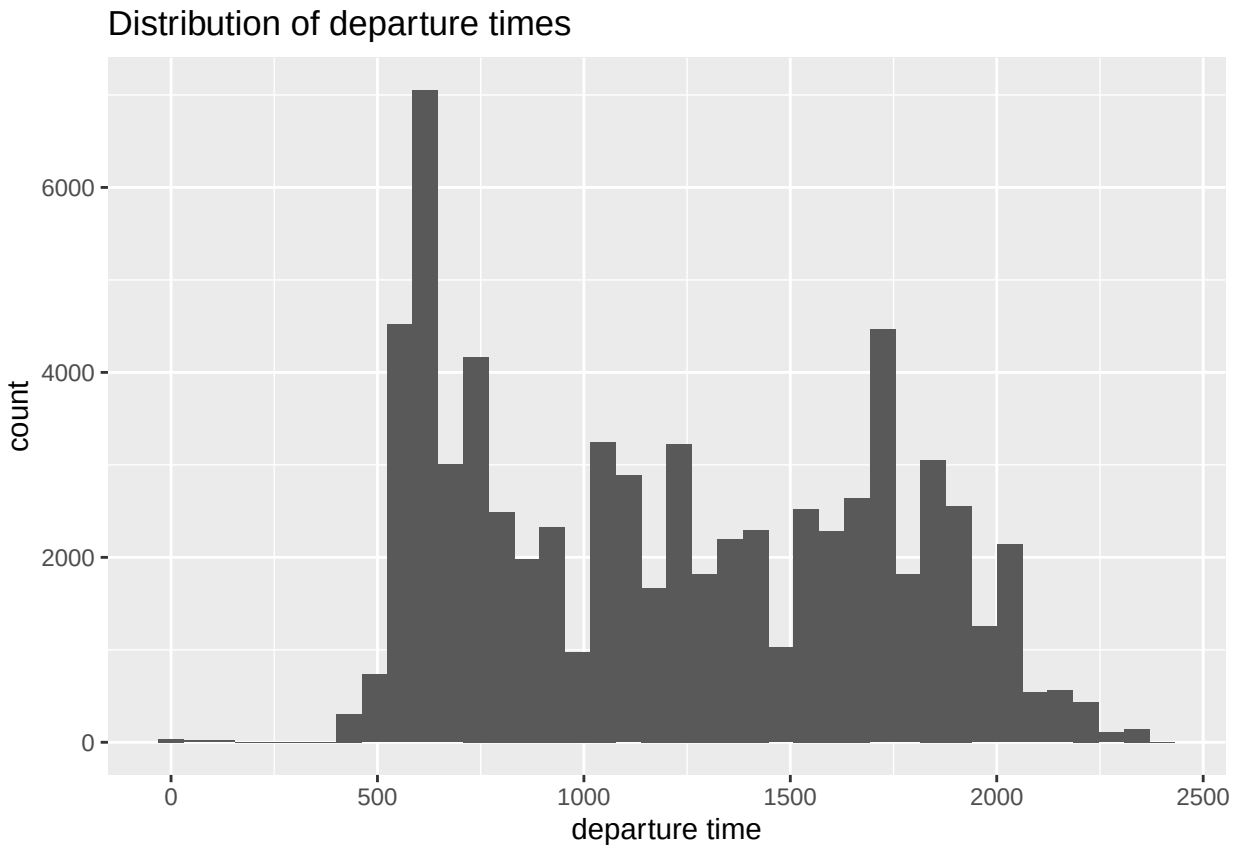
```
flights_train |>
  filter(!is.na(dep_del15)) |>
  group_by(op_unique_carrier) |>
  summarise(delay_rate = mean(dep_del15), n = n(), .groups = "drop") |>
  arrange(desc(delay_rate)) |>
  ggplot(aes(x = reorder(op_unique_carrier, delay_rate), y = delay_rate)) +
  geom_col() +
  coord_flip() +
  labs(title = "Delay rate by carrier", x = "carrier", y = "proportion delayed >= 15 min")
```



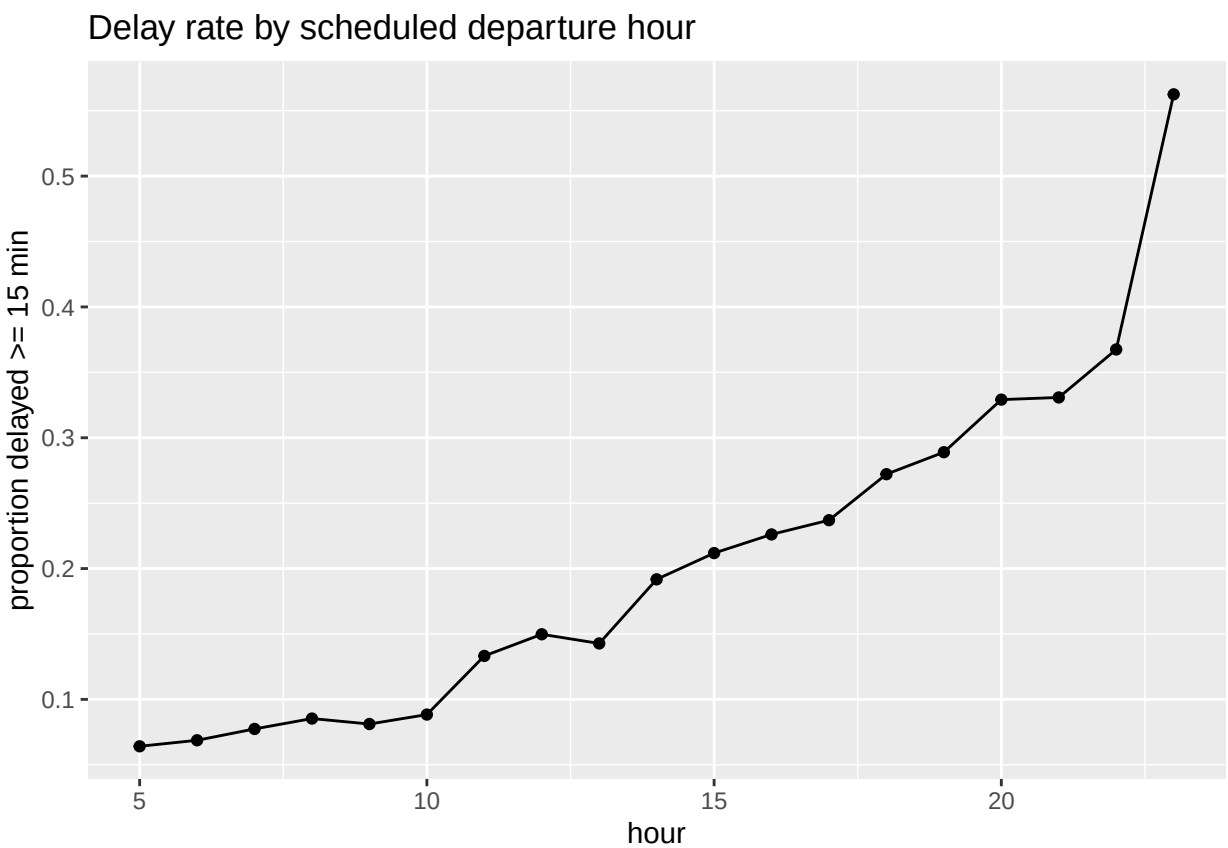
There's variation across carriers, so this should be a useful predictor. Carriers with very few flights at PIT will have noisy estimates though, so need to think about smoothing or grouping when we get to feature engineering.

Time-of-Day Delays

```
# Departure time distribution and delay rate by hour
flights_train |>
  ggplot(aes(x = dep_time)) +
  geom_histogram(bins = 40) +
  labs(title = "Distribution of departure times", x = "departure time", y = "count")
```



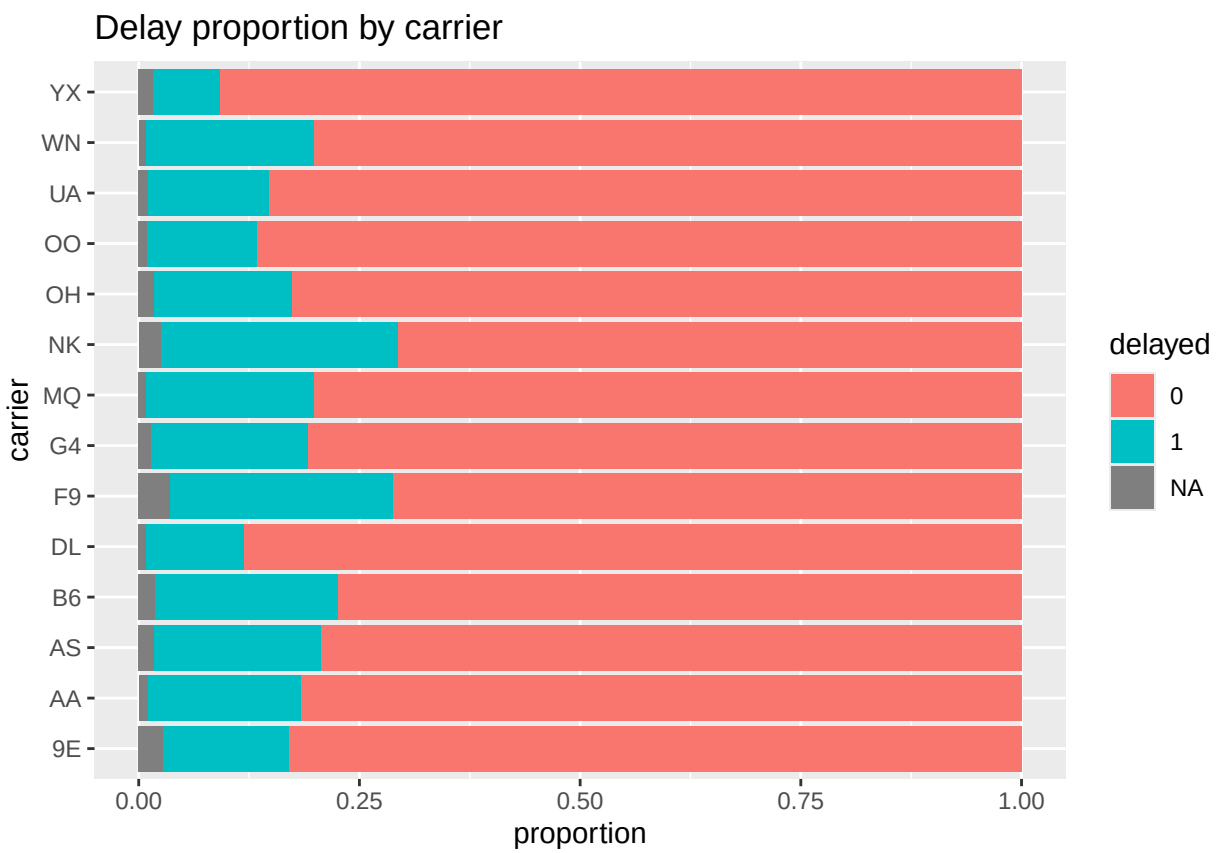

```
flights_train |>
  filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
  group_by(sched_dep_hour) |>
  summarise(delay_rate = mean(dep_del15), .groups = "drop") |>
  ggplot(aes(x = sched_dep_hour, y = delay_rate)) +
  geom_line() +
  geom_point() +
  labs(title = "Delay rate by scheduled departure hour", x = "hour", y = "proportion delayed >= 15 min")
```



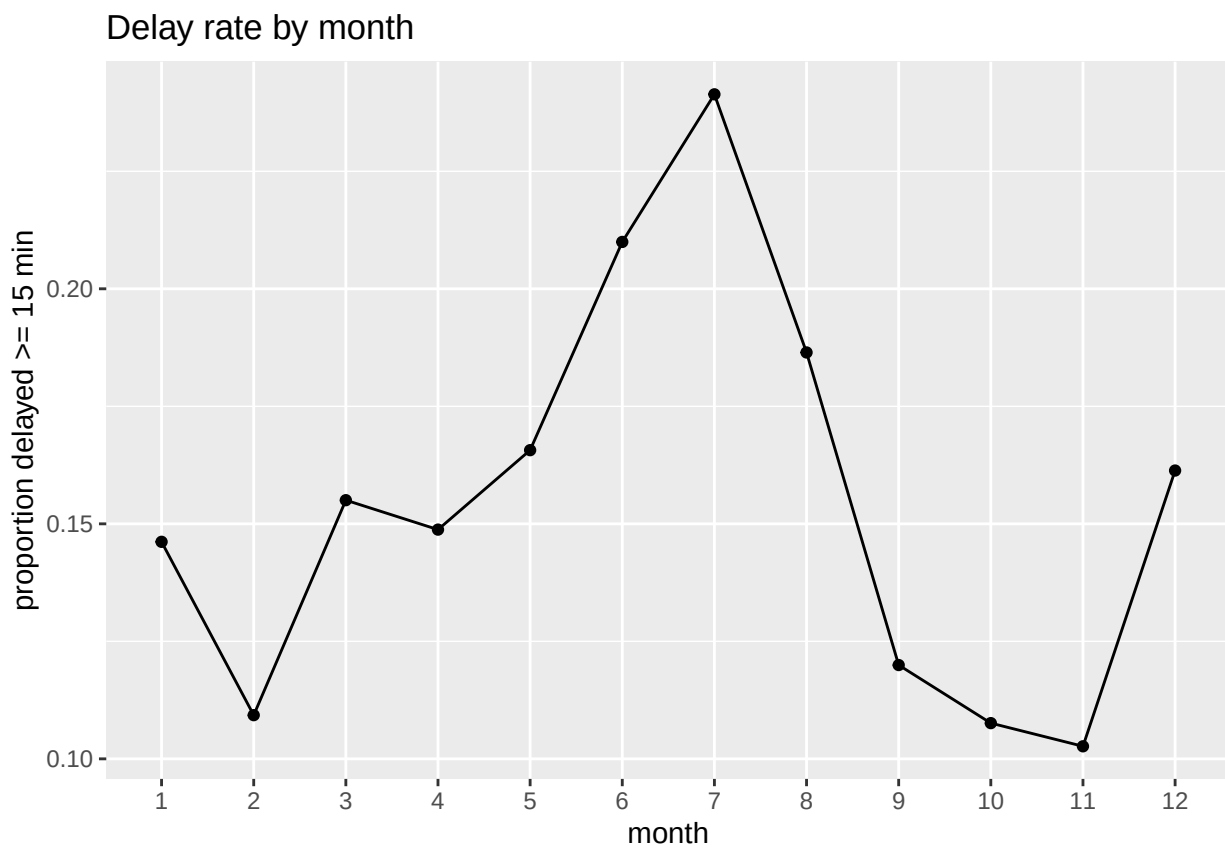
Delays increase steadily through the day and peak in the evening. This is likely due to the effect of a late plane from an earlier leg holding up the next departure. Early morning flights are the most on-time since planes are freshly repositioned overnight. `sched_dep_hour` is probably one of our most useful features.

Delay Rates

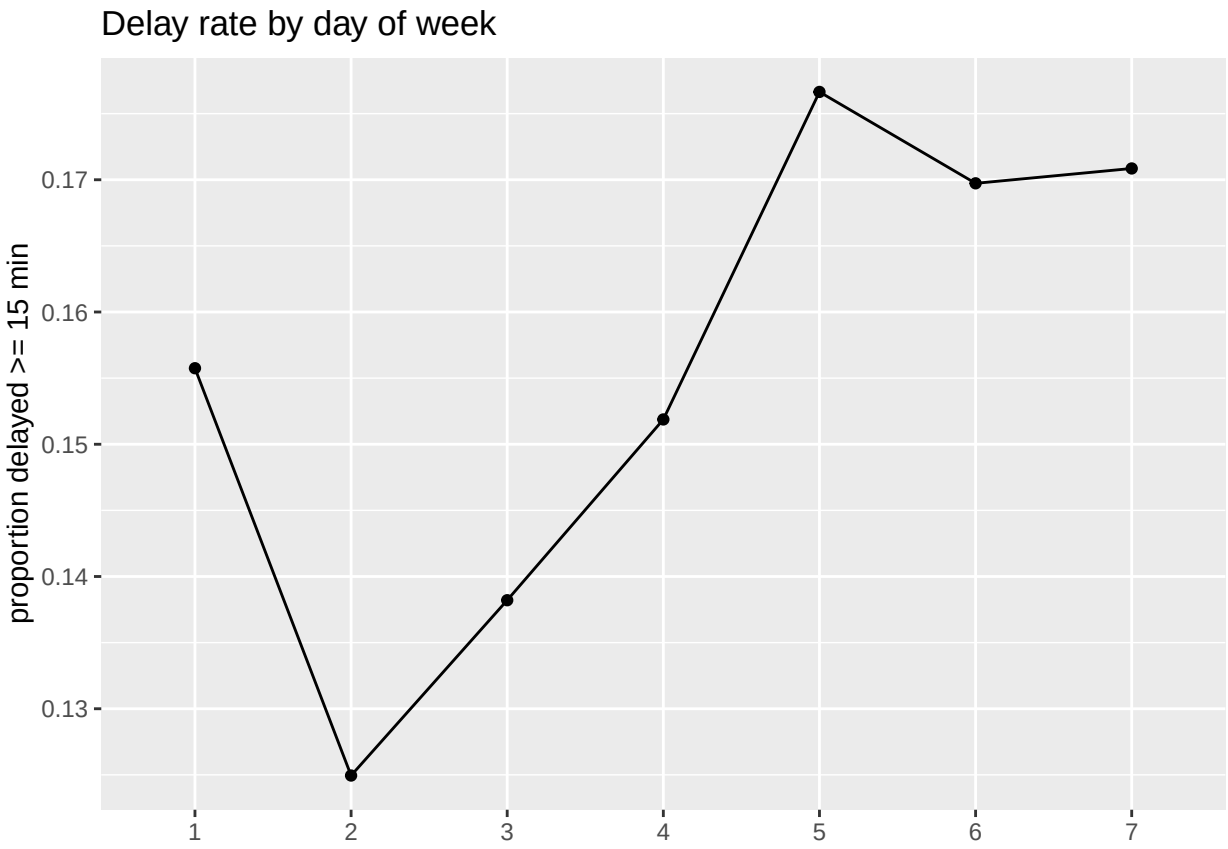
```
# Delay rate by carrier (stacked bar)
flights_train |>
  ggplot(aes(x = op_unique_carrier, fill = factor(dep_del15))) +
  geom_bar(position = "fill") +
  coord_flip() +
  labs(title = "Delay proportion by carrier", x = "carrier", y = "proportion", fill = "delayed")
```




```r
# Delay rate by month
flights_train |>
  filter(!is.na(dep_del15)) |>
  group_by(month) |>
  summarise(delay_rate = mean(dep_del15), .groups = "drop") |>
  ggplot(aes(x = month, y = delay_rate, group = 1)) +
  geom_line() +
  geom_point() +
  labs(title = "Delay rate by month", x = "month", y = "proportion delayed >= 15 min")
```

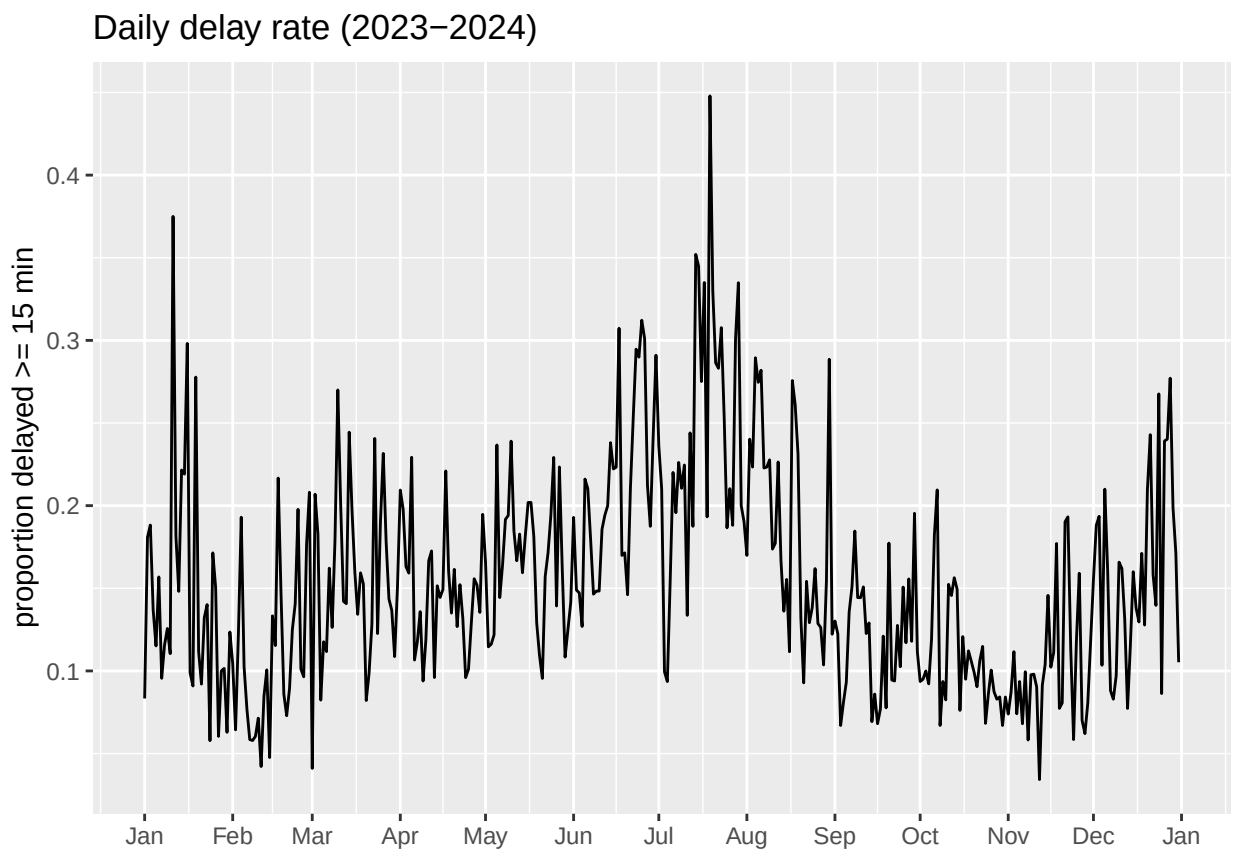



```r
# Delay rate by day of week
flights_train |>
  filter(!is.na(dep_del15)) |>
  group_by(day_of_week) |>
  summarise(delay_rate = mean(dep_del15), .groups = "drop") |>
  ggplot(aes(x = day_of_week, y = delay_rate, group = 1)) +
  geom_line() +
  geom_point() +
  labs(title = "Delay rate by day of week", x = NULL, y = "proportion delayed >= 15 min")
```



Delays are noticeably worse in summer (June–August) and December, which lines up with peak travel and weather. The day-of-week pattern is subtler — Thursdays and Fridays are slightly worse, Tuesdays are the best day to fly with the lowest proprtion of delayed flught. Both are worth including as features but probably won't carry as much weight as hour or carrier.

```
# Delay rate by date of year
flights_train |>
  filter(!is.na(dep_del15)) |>
  # fix later, but to make comparable dates
  mutate(date = make_date(year = 2010, month = month, day = day_of_month)) |>
  group_by(date) |>
  summarise(delay_rate = mean(dep_del15), .groups = "drop") |>
  ggplot(aes(x = date, y = delay_rate)) +
  geom_line() +
  scale_x_date(date_labels = "%b", date_breaks = "1 month") +
  labs(
    title = "Daily delay rate (2023-2024)",
    x = NULL,
    y = "proportion delayed >= 15 min"
  )
```



The monthly peaks are not uniform across the entire month. While it would add too many degrees of freedom to the model, it may be worth to feature-engineer high-peak indicator. It seems to overlap with academic breaks (i.e. spring break, winter break, summer break).

Top Destinations

```
# Top destinations
flights_train |>
  count(dest, sort = TRUE) |>
  slice_head(n = 15)
```

```
## # A tibble: 15 x 2
##    dest      n
##    <fct> <int>
##  1 LGA    5296
##  2 ORD    4663
##  3 ATL    4637
##  4 EWR    4417
##  5 MCO    3831
##  6 BOS    3598
##  7 CLT    3134
##  8 JFK    2939
##  9 DCA    2915
## 10 DEN    2804
## 11 DFW    2573
## 12 PHL    2553
```

```
## 13 DTW    2295
## 14 MDW    2162
## 15 MSP    2040
```

The most common routes are to major hubs (LGA, ORD, ATL, etc.), which makes sense for an airport PIT's size. Flights into busy hubs tend to be more delay-prone, so route-level delay rates could be a useful feature to engineer.